Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

Algoritmi I

Appunti a cura di
Lorenzo Ferraro

3 novembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Algoritmi I*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1 Introduzione **1**
1.1 Gli Algoritmi . . . 1
1.2 Efficienza . . . 1
1.2.1 Tempo . . . 1
1.2.2 Spazio . . . 2

2 Costo computazionale **3**
2.1 Notazione asintotica . . . 3
2.1.1 Notazione asintotica O . . . 3
2.1.2 Notazione asintotica **Ω** . . . 4
2.1.3 Notazione asintotica **Θ** . . . 5
2.1.4 Calcolare la notazione asintotica utilizzando i limiti . . . 6
2.1.5 Algebra della notazione asintotica . . . 6
2.1.6 Sommatorie notevoli . . . 7
2.2 Costo computazionale . . . 7
2.2.1 Costo computazionale e tempi di esecuzione . . . 9

3 Ricerca **11**
3.1 Il problema della ricerca . . . 11
3.2 Ricerca sequenziale . . . 11
3.3 Ricerca binaria . . . 12
3.3.1 Applicazioni della ricerca binaria . . . 13
3.4 Ricerca costante . . . 14

4 Ricorsione **15**
4.1 Algoritmi ricorsivi . . . 15
4.1.1 Meglio un'algoritmo ricorsivo o iterativo? . . . 16

Capitolo 1

Introduzione

1.1 Gli Algoritmi

La definizione di informatica proposta dall'ACM (Association for Computing Machinery) è: *"L'informatica è la scienza degli algoritmi che descrivono e trasformano l'informazione: la loro teoria, analisi, progetto, efficienza, reallizzazione e applicazione."*
Ma cos'è un algoritmo?

Definizione 1.1 – Algoritmo

Un **algoritmo** è una sequenza di comandi *elementari* e *univoci* che terminano in un tempo finito e che operano su *strutture dati*.

Un comando è *elementare* quando non può essere scomposto in comandi più semplici, mentre un comando è *univoco* quando può essere interpretato in un solo modo. Le *strutture dati* invece sono degli strumenti necessari al fine di gestire i dati veri e propri, consentendoci di leggerli e modificarli. Ci sono diverse strutture dati e non ne esiste una universale per tutti i problemi: per questo, in base al proprio problema bisogna scegliere la struttura dati più coerente.

1.2 Efficienza

1.2.1 Tempo

Un algoritmo viene sempre eseguito in un tempo finito. Ciò però non significa che tutti gli algoritmi che rilasciano un output corretto siano dei buoni algoritmi. Infatti, uno degli aspetti più importanti quando si parla di un algoritmo è la sua **efficienza**, ovvero il tempo di esecuzione dell'algoritmo e la sua memoria richiesta (il tempo di esecuzione ha però spesso la priorità). Si potrebbe, però, pensare che un supercomputer molto veloce possa bilanciare la lentezza di un algoritmo, ma ciò è errato. Vediamo perché con un esempio.
Prendiamo due computer V e L, dove V esegue 10^9 operazioni al secondo mentre L ne esegue 10^7, e abbiamo due algoritmi IS e MS, che rispettivamente richiedono di eseguire n^2 e $50n \log n$ operazioni, con n che indica la dimensione dell'input. Facciamo ora eseguire al computer veloce

V l'algoritmo più lento IS e al computer lento IS l'algoritmo più velcoe MS, ponendo come input $n = 10^6$. Vediamo come si comportanto:

- $V(IS) = \frac{2(10^6)^2}{10^9} = 2000s$
- $L(MS) = \frac{50 \cdot 10^6 \log 10^6}{10^7} = 100s$

È facile notare come il computer più lento che esegue l'algoritmo più veloce sia una soluzione ben migliore rispetto al computer più veloce che esegue l'algoritmo più lento. Inoltre, più il valore di input n aumenta, più questa differenza si fa evidente.

1.2.2 Spazio

Nell'esecuzione di un algoritmo, questo crea anche diverse risorse in memoria che occupano spazio. Questo spazio viene principalmente instanziato nella **memoria principale**, ossia la RAM. Ma, se non si ha abbastanza spazio nella RAM, si passa ad alloccare queste risorse nella **memoria secondaria**, ossia nell'SSD. Il dover accedere a questa memoria rende però molto più lento l'algoritmo, dato che ci sono sia dati sulla RAM che sull'SSD, e se c'è bisogno di fare un confronto tra loro, questo sarà lentissimo. Per questo, è necessario cercare di evitare algoritmi che consumino troppo spazio, altrimenti il rischio è quello di finire la RAM a disposizione.

Capitolo 2

Costo computazionale

2.1 Notazione asintotica

Cerchiamo ora un metodo efficace per poter confrontare due algoritmi che svolgono lo stesso lavoro, per poter capire quale dei due sia il più efficiente. Questa valutazione che faremo è utile, però, solo quando si parla di input molto grandi.

Prima di iniziare a confrontare algoritmi, bisogna conoscere la notazione asintotica.

Definizione 2.1 – Notazione asintotica

A livello matematico, la **notazione asintotica** è uno strumento che permette di confrontare il tasso di crescita di una funzione rispetto a un'altra. Esistono 3 tipi di notazione asintotica:

1) Notazione asintotica **O**, che ci permette di calcolare il limite asintotico *superiore*
2) Notazione asintotica **Ω**, che ci permette di calcolare il limtie asintotico *inferiore*
3) Notazione asintotica **Θ**, che ci permette di calcolare il limite asintotico *stretto*

N.B. Prima di passare allo studio di queste tre diverse tipologie di notazione asintotica chiariamo bene una cosa: scrivere $f(n) = O(g(n))$ e scrivere $f(n) \in O(g(n))$ è la stessa identica cosa. Discorso analogo anche per le notazioni Ω e Θ.

2.1.1 Notazione asintotica O

Definizione 2.2 – Notazione asintotica O

Date due funzioni $f(n), g(n) \geq 0$, si dice che $\boldsymbol{f(n) = O(n)}$ (o $f(n) \in O(n)$) se $\exists c > 0$ e $\exists n_0 \in \mathbb{N} : 0 \leq f(n) \leq cg(n) \ \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $\boldsymbol{f(n)}$ si trova sempre **sotto** la funzione $\boldsymbol{g(n)}$.

Esempi

$\boldsymbol{f(n) = 3n + 3}$

- $\boldsymbol{f(n) = O(n^2)}$ dato che, se prendiamo $c = 1$, $3n + 3 \leq n^2 \ \forall n \geq 4$

- $f(n) = \mathbf{O}(n^2)$ dato che, se prendiamo $c = 4$, $3n + 3 \leq 4n \ \ \forall n \geq 3$

$f(n) = \sum_{i=0}^{m} a_i n^i$ con $a_m \geq 0$ (polinomio di grado m)

$f(n) = \mathbf{O}(n^m)$ dato che $f(n) = \sum_{i=0}^{m} a_i n^i \leq \sum_{i \text{t.c.} a_i \geq 0} a_i n^i \leq n^m \sum_{i \text{t.c.} a_i \geq 0} a_i$, e se poniamo $c = \sum_{i \text{t.c.} a_i \geq 0} a_i$, troviamo proprio che $f(n) \leq cn^m \ \ \forall n \geq 1$.

Proprietà 2.1 – Regole generali della notazione asintotica O

- $\log_a n = \mathbf{O}(n^{\frac{1}{b}})$, ossia un qualsiasi logaritmo è dominato da una qualunque radice
- $n^{\frac{1}{a}} = \mathbf{O}(n^b)$ (con $a, b \geq 1$), ossia una qualsiasi radice è dominata da un qualunque polinomio
- $n^a = \mathbf{O}(b^n)$ (con $b > 1$), ossia un polinomio è dominato da un qualunque esponenziale

2.1.2 Notazione asintotica Ω

Definizione 2.3 – Notazione asintotica Ω

Date due funzioni $f(n), g(n) \geq 0$, si dice che $f(n) = \Omega(n)$ (o $f(n) \in \Omega(n)$) se $\exists c > 0$ e $\exists n_0 \in \mathbb{N} : 0 \leq cg(n) \leq f(n) \ \ \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $f(n)$ si trova sempre **sopra** la funzione $g(n)$.

Esempi

$f(n) = 2n^2 + 3$

- $f(n) = \Omega(n)$ dato che, se prendiamo $c = 1$, $n \leq 2n^2 + 3 \ \ \forall n \geq 1$
- $f(n) = \Omega(n^2)$ dato che, se prendiamo $c = 1$, $n^2 \leq 2n^2 + 3 \ \ \forall n \geq 1$

$f(n) = \sum_{i=0}^{m} a_i n^i$ con $a_m \geq 0$ (polinomio di grado m)

$f(n) = \Omega(n^m)$. Infatti, $f(n) = \sum_{i=0}^{m} a_i n^i \geq a_m n^m + \sum_{i \text{t.c.} a_i \leq 0} a_i n^i \geq a_m n^m + n^{m-1} \sum_{i \text{t.c.} a_i \leq 0} a_i$. Ponendo ora $c' = \sum_{i \text{t.c.} a_i \leq 0} a_i < 0$, troviamo che $f(n) \geq a_m n^m + c' n^{m-1}$. Abbiamo ora due possibilità:

- $a_m c' > 0$, e allora si ha $f(n) = \Omega(n^m)$ dato che il coefficiente a_m è maggiore alla somma di tutti i termini negativi, e dunque si ha che $f(n) \geq a_m n^m + c' n^{m-1} \geq a_m n^m + c' n^m = n^m(a_m + c')$, e ponendo quindi $c = a_m + c'$, troviamo che $f(n) \geq cn^m$.
- $a_m c' \leq 0$, e allora riscriviamo $a_m n^m + c' n^{m-1} = n^m(a_m + \frac{c'}{n})$. Aumentando sempre di più n, troviamo che $\frac{c'}{n}$ diventa sempre più piccolo, essendo c' costante. Non appena passiamo una certa n_0, troviamo che $\forall n \geq n_0 \;\; c = a_m + \frac{c'}{n} > 0$, e dunque troviamo $f(n) \geq cn^m \;\; \forall n \geq n_0$.

Proprietà 2.2 – Regole generali della notazione asintotica Ω

- $\boldsymbol{n^{\frac{1}{a}} = \Omega(\log_b n)}$ (con $a > 1$), ossia una qualsiasi radice domina un qualsiasi logaritmo
- $\boldsymbol{a^n = \Omega(n^b)}$ (con $a > 1$), ossia un qualsiasi esponenziale domina un qualunque polinomio

2.1.3 Notazione asintotica Θ

Definizione 2.4 – Notazione asintotica Ω

Date due funzioni $f(n), g(n) \geq 0$, si dice che $\boldsymbol{f(n) = \Theta(n)}$ (o $f(n) \in \Theta(n)$) se $\exists c_1, c_2 > 0$ e $\exists n_0 \in \mathbb{N} : c_1 g(n) \leq f(n) \leq c_2 g(n) \;\; \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $\boldsymbol{f(n)}$ si trova sempre **tra** le funzioni $\boldsymbol{c_1 g(n)}$ e $\boldsymbol{c_2 g(n)}$.
N.B. Per dimostrare che $f(n) = \Theta(g(n))$ basta dimostrare che $\boldsymbol{f(n) = O(g(n))}$ e $\boldsymbol{f(n) = \Omega(g(n))}$.

Esempi

$\boldsymbol{f(n) = 3n + 3}$

$\boldsymbol{f(n) = \Theta(n)}$, sia perché $f(n) = O(n)$ e $f(n) = \Omega(n)$, ma anche perché ponendo ad esempio $c_1 = 3$ e $c_2 = 4$ e $n_0 = 3$, troviamo che $3n \leq 3n + 3 \leq 4n + 3 \;\; \forall n \geq 3$.

$\boldsymbol{f(n) = \sum_{i=0}^{m} a_i n^i}$ con $a_m \geq 0$ (polinomio di grado m)

$\boldsymbol{f(n) = \Theta(n^m)}$ dato che $f(n) = O(n^m)$ e $f(n) = \Omega(n^m)$.

2.1.4 Calcolare la notazione asintotica utilizzando i limiti

Uno dei modi più semplici per calcolare la notazione asintotica è quello di usare i limiti. Calcolando infatti il limite

$$\lim_{n\to+\infty}\left|\frac{f(n)}{g(n)}\right|$$

abbiamo tre possibili casi:

- $\lim_{n\to+\infty}|\frac{f(n)}{g(n)}| = k \in (0,+\infty)$, e allora si può dire che $\boldsymbol{f(n) = \Theta(g(n))}$
- $\lim_{n\to+\infty}|\frac{f(n)}{g(n)}| = +\infty$, e allora si può dire che $\boldsymbol{f(n) = \Omega(g(n))}$
- $\lim_{n\to+\infty}|\frac{f(n)}{g(n)}| = 0$, e allora si può dire che $\boldsymbol{f(n) = O(g(n))}$

2.1.5 Algebra della notazione asintotica

Proprietà 2.3 – Regole della notazione asintotica

N.B. Queste regole valgono per tutte le 3 notazioni, nonostante verrà usata solo la notazione O per gli esempi.

1) $\forall k > 0$ e $\forall f(n) \geq 0$, $\boldsymbol{f(n) = O(g(n)) \implies kf(n) = O(g(n))}$
2) $\forall f(n), d(n) \geq 0$, $\boldsymbol{f(n) = O(g(n))}$ e $\boldsymbol{d(n) = O(h(n)) \implies f(n) + d(n) = O(g(n)(h(n))) = O(\max\{g(n), h(n)\})}$
3) $\forall f(n), d(n) \geq 0$, $\boldsymbol{f(n) = O(g(n))}$ e $\boldsymbol{d(n) = O(h(n)) \implies f(n)d(n) = O(g(n)h(n))}$

Dimostrazione

Dimostreremo queste tre regole con i limiti, ma possono essere dimostrati anche con la definizione generale.

1) Se $f(n) = O(g(n))$, allora $\lim_{n\to+\infty}\frac{f(n)}{g(n)}$. Se ora moltiplichiamo questo limite per k, notiamo che il limite rimane 0, dunque anche $kf(n) = O(g(n))$. Procedimento analogo per le notazioni Ω e Θ. ■
2) Se $f(n) = O(g(n))$ e $d(n) = O(h(n))$, allora $\lim_{n\to+\infty}\frac{f(n)}{g(n)} = 0$ e $\lim_{n\to+\infty}\frac{d(n)}{h(n)}$. Ponendo ora $k(n) = \max\{g(n), h(n)\}$, sappiamo che sicuramente $\lim_{n\to+\infty}\frac{f(n)}{k(n)} = \lim_{n\to+\infty}\frac{d(n)}{k(n)} = 0$. Dunque $\lim_{n\to+\infty}\left(\frac{f(n)}{k(n)} + \frac{d(n)}{k(n)}\right) = \lim_{n\to+\infty}\frac{f(n)+d(n)}{k(n)} = 0$, ossia $f(n) + d(n) = O(k(n)) = O(\max\{g(n), h(n)\})$. Procedimento analogo per le notazioni Ω e Θ. ■
3) Se $f(n) = O(g(n))$ e $d(n) = O(h(n))$, allora $\lim_{n\to+\infty}\frac{f(n)}{g(n)} = 0$ e $\lim_{n\to+\infty}\frac{d(n)}{h(n)}$. Dunque $\lim_{n\to+\infty}\frac{f(n)d(n)}{g(n)h(n)} = 0$, ossia $f(n)d(n) = O(g(n)h(n))$. Proceidmento analogo per le notazioni Ω e Θ. ■

2.1.6 Sommatorie notevoli

Proprietà 2.4 – Sommatorie notevoli

1) $\sum_{i=0}^{n} i = \Theta(n^2)$ o più in generale $\sum_{i=0}^{n} i^c = \Theta(n^{c+1}) \ \forall c \in \mathbb{N}$
2) $\sum_{i=0}^{n} 2^i = \Theta(2^n)$ o più in generale $\sum_{i=0}^{n} c^i = \Theta(c^n) \ \forall c > 1$ e $\sum_{i=0}^{n} c^i = \Theta(1) \ \forall c < 1$
3) $\sum_{i=0}^{n} i2^i = \Theta(n2^n)$ o più in generale $\sum_{i=0}^{n} ic^i = \Theta(nc^n) \ \forall c > 1$ e $\sum_{i=0}^{n} ic^i = \Theta(1) \ \forall c < 1$
4) $\sum_{i=1}^{n} \log i = \Theta(n \log n)$ o più in generale $\sum_{i=1}^{n} \log_c i = \Theta(n \log_c n) \ \forall n \in \mathbb{N}$
5) $\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$

Dimostrazione

1) $\sum_{i=0}^{n} i^c \approx \int_0^n x^c \, dx = \left[\frac{x^{c+1}}{c+1}\right]_0^n = \frac{n^{c+1}}{c+1} = \Theta(n^{c+1})$. ■
2) Ricordando la serie geometrica $\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}$, $\sum_{i=0}^{n} c_i = \Theta(c^{n+1})$. Se però $c < 1$, allora $\lim_{n\to+\infty} \frac{c^{n+1}-1}{c-1} = -\frac{1}{c-1}$. Questo valore è un costante positiva K, dunque troviamo che $\sum_{i=0}^{n} c^i < K$, ossia $\sum_{i=0}^{n} c^i = \Theta(1)$. ■
3) Ricordando la serie geometrica $\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}$, se deriviamo entrambi i membri troviamo $\sum_{i=0}^{n} ic^{i-1} = \frac{(n+1)c^n(c-1)-(c^{n+1}-1)}{(c-1)^2} = \frac{nc^{n+1}-nc^n-c^n+1}{(c-1)^2}$, e moltiplicando entrambi i membri per c troviamo che $\sum_{i=0}^{n} ic^{i-1} = \frac{nc^{n+2}-nc^{n+1}-c^{n+1}+}{(c-1)^2}$. Se studiamo il limite con $n \to +\infty$, notiamo come il temrine che vinca sia effettivamente $nc^{n+2} = nc^n c^2$, e quindi troviamo che $\lim_{n\to+\infty} \sum_{i=0}^{n} ic^i = \Theta(nc^n)$. Se però $c < 1$, allora $\lim_{n\to+\infty} c^{n+1} = 0$ e $\lim_{n\to+\infty} nc^{n+2} = 0$. Dunque $\lim_{n\to+\infty} \sum_{i=0}^{n} ic^i = \frac{c}{(1-c)^2} = K$, ossia $\sum_{i=0}^{n} ic^i = \Theta(1)$. ■
4) $\sum_{i=1}^{n} \log_c n \approx \int_1^n \log_c x \, dx = \left[x \log_c x - \frac{x}{\ln x}\right]_1^n = n \log_c n - \frac{n}{\lg c} - \log_c 1 + \frac{1}{\ln c}$. Studiando il limite, troviamo che $\lim_{n\to+\infty} \sum_{i=1}^{n} \log_c n = \lim_{n\to+\infty} n \log_c n - \frac{n}{\ln c} - \log_c 1 + \frac{1}{\ln c} = \lim_{n\to+\infty} n \log_c n$, dunque troviamo che $\sum_{i=1}^{n} \log_c n = \Theta(n \log_c n)$. ■
5) $\sum_{i=1}^{n} \frac{1}{i} \approx \int_1^n \frac{1}{x} \, dx = [\ln x]_1^n = \ln n - \ln 1 = \ln n = \frac{\log n}{\log e} = \Theta(\log e)$ dato che $\frac{1}{\log e}$ è una costante. ■

2.2 Costo computazionale

Definizione 2.5 – Costo computazionale

Il **costo computazionale** di un'istruzione è la quantità di risorse (spazio o tempo, noi quasi sempre parleremo di costo computazionale temporale) impiegato dal processore per eseguirla.

In base al tipo di istruzioni, queste hanno tutte un costo computazionale diverso.

- **Operazioni elementari**: hanno come costo computazionale $\Theta(1)$, ossia costo *costante* che non dipende dall'input, tra cui ci sono ad esempio operazioni aritmetiche, la lettura del valore di una variabile, la lettura della lunghezza di un array (che in python non viene calcolato ogni volta ma è un dato salvato) e così via.
- **Istruzioni if/else**: hanno computazionali costo pari a: il **costo di verifica della condizione** (che spesso è però costante) + **il massimo dei costi tra le istruzioni dell'if e quelle dell'else**.
- **Istruzioni iterative (cicli)**: hanno costo computazionale pari alla **somma dei costi di ciascuna iterazione**. Se tutte le iterazioni hanno lo stesso costo, il costo è pari al **prodotto del costo di una singola iterazione per il numero di iterazioni**. La condizione viene però valutata sempre una volta in più rispetto al numero di iterazioni, perché per uscire dal ciclo è necessario che l'ultimo controllo risulti negativo.

Definizione 2.6 – Costo computazionale di un algoritmo

Il **costo computazionale di un algoritmo** ($T(n)$) è pari alla somma dei costi delle istruzioni che lo compongono.

Un algoritmo può avere tempi di esecuzioni differenti in base all'input. Parliamo in questo caso di costo computazionale migliore e peggiore.

Definizione 2.7 – Caso migliore e peggiore

Fissata una dimensione di input, prendiamo per calcolare il **caso migliore** l'input più favorevole al nostro algoritmo, mentre per il **caso peggiore** prendiamo quello più sfavorevole.

Esempi

N.B. Tutti i codici scritti in questi appunti sono in *Python/Pseudocodice*.

```
def Calcolo_Polinomio(A,c):
    somma = A[0] # Costo costante
    potenza = 1 # Costo costante
    for i in range(1, len(A)): # n-1 iterazioni
        potenza = c * potenza # Costo costante
        somma += A[i] * potenza # Costo costante
    return somma # Costo costante
```

Il costo computazionale totale è quindi dato dalla somma di tutti i costi delle singole operazioni e dal costo del ciclo for. Troviamo quindi che $T(n) = \Theta(1) + (n-1)\Theta(1) = \Theta(1) + \Theta(n) = \Theta(n)$.

```python
def Esercizio(n):
    n = abs(n) # Costo costante
    t = s = n # Costo costante
    p = 0 # Costo costante
    while s > 1: # ? iterazioni
        p += 1 # Costo costante
        s /= 4 # Costo costante
    while n - p > 0: # ? iterazioni
        n -= p # Costo costante
        t += 5 # Costo costante
    return p # Costo costante
```

Studiamo il primo while e vediamo come varia il valore di s e di p col passare delle iterazione per capire il numero di queste.

it.	$1°$	$2°$	$\dots$	$k°$
s	$\frac{n}{4}$	$\frac{n}{16}$	$\dots$	$\frac{n}{4^k}$
p	1	2	$\dots$	k

Dato che per uscire dal while è necessario che $s = 1$, poniamo $\frac{n}{4^k} = 1$. Troviamo quindi che $n = 4^k$, ossia $k = \log_4 n$. Dunque in questo while faremo $\log_4 n$ giri, ossia circa $\log n$ iterazioni, per un costo totale di $\Theta(\log n)$.
Studiamo ora il secondo while e vediamo come varia la variabile n per capire dopo quante iterazioni usciremo dal while.

it.	$1°$	$2°$	$\dots$	$k°$
n	$n-p$	$n-2p$	$\dots$	$n-hp$

Dato che per uscire dal while è necessario che $n - p = 0$, alla h-esima iterazione troviamo che $n - hp - p = 0$, ossia $n - p(h+1) = 0$. Ricordando che $p = k = \log_4 n$, troviamo che $n - \log_4 n(h+1) = 0$, ossia $h = \frac{n}{\log_4 n} - 1$. Dunque in questo while faremo $\frac{n}{\log_4 n} - 1$ giri, ossia circa $\frac{n}{\log n}$ iterazioni, per un costo totale pari a $\Theta(\frac{n}{\log n})$.

Facendo ora la somma tra tutti i costi computazionali, troviamo che $\boldsymbol{T(n) = \Theta(1) + \Theta(\log n) + \Theta(\frac{n}{\log n}) = \Theta(\frac{n}{\log n})}$ (questo perché $\frac{n}{\log n}$ cresce molto più velocemente di $\log n$, dato che $n >> \log n$).

2.2.1 Costo computazionale e tempi di esecuzione

Vediamo ora a livello pratico quanto il costo computazionale possa modificarne i tempi di esecuzione.

Prendiamo un calcolatore in grado di eseguire un'operazione elementare in un nanosecondo (riuscendo quindi ad eseguire 10^9 operazioni al secondo), e prendiamo un input di $n = 10^6$. Vediamo come si comportano alcuni algoritmi con diversi costi computazionali:

- $O(n)$: tempo di esecuzione = 1 millessimo di secondo
- $O(n \log n)$: tempo di esecuzione = 20 millessimi di secondo
- $O(n^2)$: tempo di esecuzione: 16 minuti e 40 secondi
- $O(2^n)$: tempo di esecuzione = circa $9,9 \cdot 10^{301029}$ secondi

È palese dunque l'enorme differenza che vi è tra i diversi costi computazionali: in particolare, il passaggio da $O(n^2)$ e $O(2^n)$ risulta molto drastico.

Dato che il tempo di esecuzione di $9,9 \cdot 10^{301029}$ è un tempo assurdo, proviamo a prendere un calcolatore più veloce per vedere se riusciamo a migliorare la situazione.
Stabiliamo un tempo arbitrario T e confrontiamo due calcolatorI: C_1 che è in grado di eseguire 10^k operazioni al secondo e C_2 che è in grado di eseguirne 10^{k+3}. Dunque C_2 è 1000 volte più veloce di C_1. Prendiamo un input id $n = 1000$ per il calcolatore più lento e di $n = 1000 + x$ per il calcolatore più veloce. Dunque $T = \frac{2^{1000}}{10^k} = \frac{2^{1000+x}}{10^{k+3}}$, ossia $\frac{2^{1000+x}}{2^{1000}} = 2^x = \frac{10^{k+3}}{10^k} = 10^3$, dunque $x = \log(10^3) \approx 10$. Dunque prendendo un calcolatore 1000 volte più veloce riesce, ad aumentare l'input solo da 1000 a 1010: una differenza irrisoria. Risulta quindi evidente come un algoritmo con costo pari a 2^n sia un algoritmo veramente poco efficiente, soprattutto con input grandi.

Capitolo 3

Ricerca

3.1 Il problema della ricerca

Il problema della ricerca di un elemento in un insieme di dati in informatica è uno dei problemi storici più studiati, essendo molto comune. È utile quindi cecare un algoritmo che risolva questo problema nel modo più efficeinte possibile.

In breve, questo problema consiste nel ricevere in input una serie di dati, come ad esempio un array `A` di n valori, e un valore `v` da cercare. In output bisogna restituire l'indice `i` tale che `A[i] = v`, oppure un valore `None` o `-1` che indica che `v` non è presente nell'array.

3.2 Ricerca sequenziale

Definizione 3.1 – Ricerca sequenziale

Il metodo più intuitivo per risolvere questa tipologia di problemi è la **ricerca sequenziale**. L'idea è semplice:

1° Si ispeziona uno per uno gli elementi dell'array;

2° li si confronta tutti con il valore `v`;

3° si restituisce l'indice `i` appena si trova `v` (se lo si trova, altrimenti si restituisce `None`/`-1`).

Il codice è molto semplice:

```
def Ricerca_Sequenziale(A,v):
    i = 0
    while ((i < len(A)) and (A[i] != v)):
        i += 1
    if (i < len(A)):
        return i
    return None
```

Il costo computazionale qui dipende da dove si trova `v` all'interno dell'array. Nel **caso migliore**, ossia quando `v` si trova alla prima posizione dell'array, il costo è $\mathbf{\Theta(1)}$, mentre nel **caso peggiore**,

ossia quando v non è presente nell'array, abbiamo invece un costo di $\Theta(n)$. In generale, possiamo quindi dire che l'algoritmo ha un costo di $O(n)$: questo significa che per alcuni input il costo è di $\Theta(n)$, ma non è sempre così.

Definizione 3.2 – Caso medio

Per trovare il **caso medio** di un algoritmo, bisogna calcolare la probabilità che ogni caso possibile si verifichi e moltiplicarla per il costo computazionale di quel caso specifico. Per rendere più semplici i calcoli, spesso si generalizzano tutti i casi indicandoli con la stessa probabilità p e lo stesso costo computazionale $T(n)$. Dunque

$$T_{\text{medio}}(n) = p \sum_{i=1}^{k} T(n)$$

Nel caso della ricerca sequenziale, il costo nel caso medio è pari a $T_{\text{medio}}(n) = \frac{1}{n} \sum_{i=0}^{n-1} i = \frac{1}{n} \cdot \frac{n(n-2)}{2} = \frac{n-1}{2} = \Theta(n)$, pari al caso peggiore.

3.3 Ricerca binaria

Definizione 3.3 – Ricerca binaria

Un algoritmo migliore per risolvere questo problema è la **ricerca binaria**, che richiede però che la lista parta **ordinata**. Anche qui l'idea è semplice:

1° Si ispeziona l'elemento centrale della lista;

2° se è uguale al valore cercato ci si ferma e si restituisce l'indice `i`;

3° altrimenti si ripete lo stesso procedimento prendendo solo una metà della lisa: la metà sinistra se il valore centrale è più grande di `v`, o quella destra.

Vediamo il codice:

```
def Ricerca_Binaria(A,v):
    a,b = 0
    m = (a+b)//2
    while (A[m] != v):
        if (A[m] > v):
            b = m - 1
        else:
            a = m + 1
        if (a > b):
            return None
        m = (a+b)//2
    return m
```

Anche qui il costo computazionale dipende da dove si trova `v` all'interno dell'array. Nel **caso migliore**, ossia quando `v` è l'elemento centrale dell'array, il costo è pari a $\mathbf{\Theta(1)}$. Nel **caso peggiore** invece, ossia quando `v` non è proprio presente nell'array, il costo è pari a $\mathbf{\Theta(\log n)}$, perché ad ogni giro la dimensione dell'array viene dimezzata, dunque per leggerlo tutto bastano $\log n$ iterazioni, con un costo ognuna di $\Theta(1)$. In generale, possiamo quindi dire che il costo della ricerca binaria è pari a $\mathbf{O(\log n)}$.

Dato che però caso migliore e peggiore anche in questo caso sono diversi, cerchiamo di studiare il caso medio.
Partiamo col porre alcune condizioni che non cambiano in alcun modo il risultato finale, ma che ci facilitano molto i calcoli:

- Il numero di elementi nell'array è una potenza di 2
- `v` è presente nella lista
- Tutte le posizioni tra 1 e n sono equiprobabili.

Chiediamoci ora a ogni iterazioni quante posizioni dell'array stiamo effettivamente controllando. Alla prima iterazione controlliamo solo la posizione `n//2`, ossia solo 1 posizione. Alla seconda invece controlliamo `n//4` se `v < n//2` e `3n//4` se `v > n//2`, dunque 2 posizioni controllate. Alla terza iterazione controlliamo `n//8, 3n//8, 5n//8, 7n//8`, ossia 4 posizioni e così via. Alla i-esima iterazione controlliamo sempre 2^{i-1} posizioni. La probabilità quindi che l'algoritmo termini all'iterazione i è $\frac{2^{i-1}}{n}$, dunque il costo computazionale medoi è $\boldsymbol{T_{\text{medio}}(n)} = \sum_{i=1}^{\log n} \Theta(i)\frac{2^{i-1}}{n} = \Theta((\log n - 1)2^{\log n})\frac{1}{n} = \Theta(\frac{(\log n - 1)2^{\log n}+1}{n}) = \mathbf{\Theta(\log n)}$, pari al costo nel caso peggiore.

3.3.1 Applicazioni della ricerca binaria

Analizziamo il seguente esercizio:
"Dato un array `A` di interi, ordinato in modo crescente e senza duplicati, e dati due valori `a` e `b` tali che $a \leq b$, si vuole sapere **quanti elementi di `A` sono compresi nell'intervallo `[a, b]`**." Vediamo come risolverlo.

Per risolvere questo esercizio possiamo sfruttare la ricerca binaria: essendo infatti l'array ordinato, il numero di elementi di `A` compresi tra `a` e `b` è calcolabile facilmente se si conoscono le posizioni di `a` e `b` in `A`. Infatti, il numero di elementi sarà pari a $ind_B - ind_A + 1$. Vediamo come si traduce questo in codice:

```
1 def Applicazione_Ricerca_Binaria(A,a,b):
2     ind_a = Ricerca_Binaria(A,a)
3     ind_b = Ricerca_Binaria(A,b)
4     if A[ind_A] < a:
5         ind_A += 1
```

```
6       if A[ind_B] > b:
7           ind_B -= 1
8       return ind_b - ind_A + 1
```

I due controlli sugli indici di `a` e `b` sono fondamentali: se infatti non dovessero essere presenti `a` o `b` nell'array, è importante non prendere nell'intervallo elementi più piccoli di `a` o più grandi di `b`.

Il costo computazionale di questo algoritmo è pari a $\boldsymbol{\Theta(\log n)}$ in quasi tutti i casi. Quasi tutti perché in realtà esiste il caso migliore dove `a` e `b` vengono trovati al primo/secondo tentativo, ma è molto raro come caso (significherebbe trovare `a` all'indice $n//2$ e `b` all'indice $n//4$ o $3n//4$, o viceversa), che è un caso molto raro.

3.4 Ricerca costante

L'ultimo algoritmo di ricerca di cui parliamo è la **ricerca costante**. Un algoritmo informatico che riesca a risolvere il problema della ricerca con un costo costante $\Theta(1)$ attraverso dei confronti purtroppo non esiste, ma sperimentando con il DNA si è scoperto un algoritmo simile in natura. Infatti, si è visto come una sonda in un frammento di DNA si riesca ad attaccare alla sua sequenza complementare senza perdere tempo e senza curaarsi della lunghezza del DNA, con un costo computazionale dunque costante.

Capitolo 4

Ricorsione

4.1 Algoritmi ricorsivi

Studiamo un algoritmo alternativo che esegue la ricerca binaria.

```
def Ricerca_Binaria_Ricorsiva(A,v,a,b):
    if b is None:
        b = len(A) - 1
    if a > b:
        return None
    m = (a+b)//2
    if A[m] == v:
        return m
    if v < A[m]:
        return Ricerca_Binaria_Ricorsiva(A,v,a,m-1)
    else:
        return Ricerca_Binaria_Ricorsiva(A,v,m+1,b)
```

La cosa più interessante di questo algoritmo è che a tutti gli effetti *richiama* sé stesso. Infatti se non ci troviamo nel caso dove `v == m`, allora l'algoritmo viene richiamato prendendo però solo la parte o destra o sinistra dell'array. Stiamo, dunque, utilizzando un algoritmo **ricorsivo**.

Definizione 4.1 – Funzioni e algoritmi ricorsivi

Una funzione matematica è detta **ricorsiva** se la sua definizione è espressa in termini di sé stessa. Questa deve però sempre avere un caso base.
Un algoritmo è detto **ricorsivo** se è espresso in termini di sé stesso.
N.B. Un qualsiasi algoritmo ricorsivo può essere riscritto come un algoritmo iterativo.

Un algoritmo ricorsivo è quindi un algoritmo la cui soluzione è costruita risolvendo uno o più sottoproblemi di dimensione minore e unendone le soluzioni. Questi sottoproblemi convergono in un sottoproblema che fa da **caso base**, du cui si conosce a priori la soluzione senza dover studiare nessun sottoproblema.

Esempio

Scrivere un algoritmo ricorsivo per **calcolare il fattoriale di un numero intero n**.

Vediamo prima il codice per poi studiarlo.

```
def Fattoriale_Ricorsivo(n):
    if n == 1:
        return 1
    return n * Fattoriale_Ricorsivo(n-1)
```

L'algoritmo sfrutta la definzione ricorsiva del fattoriale, dove infatti $n! = n \cdot (n-1)!$, per suddividere il problema in sottoproblemi. Il caso base in questo caso è quando `n == 1`, dato che $1! = 1$.

4.1.1 Meglio un'algoritmo ricorsivo o iterativo?

Dato che tutti gli algoritmi ricorsivi possono essere riscritti come algoritmi iterativi, quando bisognerebbe scegliere un algoritmo iterativo e quando uno ricorsivo? Dipende molto dalla situazione.

- Meglio un algoritmo **iterativo** se:
  - è più efficiente
  - la soluzione iterativa è semplice e chiara tanto quanto quella ricorsiva
- Meglio un algoritmo **ricorsivo** se:
  - è più efficiente
  - la formulazione del problema fa subito pensare a una risoluzione ricorsiva

Quando parliamo di efficienza in questo caso non parliamo strettamente di efficienza in termini di tempo, bensì in termini di **memoria**.

Definizione 4.2 – Esigenze di memoria

Un'algoritmo ricorsivo avrà praticamente sempre **maggiori esigenze in termini di memoria** rispetto ai corrispettivi algoritmi iterativi, dato che lascia in sospeso molte chiamate aspettando che le successive vengano risolte.

Se pensiamo ad esempio all'algoritmo del calcolo del fattoriale, per calcolare 10! dobbiamo prima calcolare 9!, ma per risovlere 9! dobbiamo prima calcolare 8! e così via, fino ad arrivare al caso base. In totale, vi sono 9 chiamate in sospeso in attesa che le successive vengano risolte.

Esempi

Scrivere un algoritmo ricorsivo per **calcolare l'n-esimo numero di Fibonacci**.

Ricordiamo la definizione di numero di Fibonacci: $F(0) = F(1) = 1$ e $F(i) = F(i-1) + F(i-2)$ se $i > 1$.

```
def Fibonacci(n):
    if n <= 1:
        return 1
    return Fibonacci(n-1) + Fibonacci(n-2)
```

Notiamo come ci sono ben due chiamate ricorsive nel nostro programma. Questo renderà più complessa e dispendiosa la risoluzione del problema con input molto grandi, dato che genera un numero esponenziale di sottoproblemi.

Osservazione 4.1 – Osservazioni sull'algoritmo ricorsivo di Fibonacci

1) **Lo stesso calcolo viene effettuato più volte** (calcoliamo `Fibonacci(n-2)` sia per calcolare `Fibonacci(n)` che per calcolare `Fibonacci(n-1)` ad esempio).
2) **Occupiamo tanta memoria**, perché per poter calcolare `Fibonacci(n)` è necessario un numero di sottoproblemi elevatissimo (2^{n-1}), come avevamo già notato.
3) Per risolvere il problema di dimensione n bisogna risolvere **due problemi di dimensione molto simile** ($n-1$ e $n-2$).

Il costo computazionale di questo algoritmo non sappiamo ancora come calcolarlo (sono necessarie le equazioni di ricorrenza che vedremo più avanti), ma possiamo già notare una cosa. Se $n \leq 1$, allora il costo è $T(0) = T(1) = \Theta(1)$, altrimenti abbiamo un costo $T(n) = \Theta(1) + T(n-1) + T(n-2)$. Questa formula è molto simile alla definizione dei numeri di Fibonacci, che hanno un andamento esponenziale. Possiamo quindi aspettarci che anche l'algoritmo ricorsivo di Fibonacci abbia un **costo esponenziale**.

Vediamo brevemente anche l'algoritmo iterativo per calcolare il numero di Fibonacci.

```
def Fibonacci_iterativo(n):
    if n <= 1:
        return 1
    fib_2 = 0
    fib_1 = 1
    for i in range(2, n+1):
        fib_2, fib_1 = fib_1, fib_2 + fib_1
    return fib_1
```

Questo algoritmo ha un costo in termini di tempo pari a $\Theta(n)$, e in termini di memoria pari a $\Theta(1)$.

Scrivere un algoritmo ricorsivo per **risolvere il problema delle Torri di Hanoi**.